

Syntropy Goal Engine

Para quê a Syntropy age.

Todos os outros motores respondem “como”. O Goal Engine responde “para quê”. Ele define a hierarquia de objetivos que orienta cada decisão. Sem ele, a Syntropy seria inteligente mas sem propósito — capaz de agir, mas sem saber o que priorizar.

O Problema

A Syntropy frequentemente enfrenta trade-offs:

Opção A	vs	Opção B
Aumentar faturamento		Reduzir fila
Maximizar margem		Maximizar satisfação
Vender tudo		Evitar ruptura
Promoção agressiva		Manter preço premium
Atender rápido		Atender bem

Sem uma hierarquia clara de objetivos, cada decisão seria arbitrária. O Goal Engine elimina essa ambiguidade.

Hierarquia de Objetivos

A Syntropy segue esta ordem de prioridade. Em caso de conflito, o objetivo superior vence:

NÍVEL 0 – SEGURANÇA (inviolável)

"Nunca causar dano ao evento ou às pessoas"

- Não permitir fraude
- Não causar ruptura total (evento parar)
- Não expor dados pessoais
- Não tomar ação irreversível sem confiança alta

NÍVEL 1 – EXPERIÊNCIA DO PARTICIPANTE

"O participante deve ter uma experiência fluida"

- Fila aceitável (dentro do contexto)
- Produtos disponíveis
- Pedido entregue no tempo prometido
- Comunicação clara

NÍVEL 2 – LUCRO DO PRODUTOR

"O produtor deve ganhar o máximo possível"

- Maximizar faturamento
- Maximizar margem líquida
- Reduzir desperdício
- Otimizar custos operacionais

NÍVEL 3 – EFICIÊNCIA OPERACIONAL

"A operação deve rodar com mínimo de fricção"

- Equipe bem distribuída
- Estoque balanceado
- Tempo de preparo otimizado
- Comunicação interna fluida

NÍVEL 4 – APRENDIZADO

"Cada evento deve gerar experiência operacional"

- Coletar dados para melhorar previsões
- Testar hipóteses (A/B de preços, promoções)
- Calibrar modelos
- Expandir knowledge graph

Como a Hierarquia Resolve Conflitos

Conflito 1: Faturamento vs Fila

Situação: Promoção pode aumentar faturamento em R\$ 5.000, mas vai gerar fila de 20min.

Análise:

Nível 2 (Lucro): +R\$ 5.000 ✓

Nível 1 (Experiência): Fila 20min > limite aceitável ✗

Decisão: NÃO fazer promoção agora.

Razão: Experiência (Nível 1) > Lucro (Nível 2)

Alternativa: Esperar fila cair para < 8min, depois ativar.

Conflito 2: Margem vs Satisfação

Situação: Aumentar preço da cerveja em R\$ 3 no pico aumenta margem em 15%, mas pode irritar público.

Análise:

Nível 2 (Lucro): +15% margem ✓

Nível 1 (Experiência): Preço justo = satisfação ✓/✗

Decisão: DEPENDE DO CONTEXTO

- Festival popular: NÃO (público sensível a preço)
- Evento VIP: SIM (público aceita premium)
- Corporativo: SIM (empresa paga, não o participante)

Conflito 3: Vender tudo vs Evitar ruptura

Situação: Estoque de Gin Tônica para 2h.

Promoção pode vender tudo em 1h.

Análise:

Nível 2 (Lucro): Vender tudo = receita máxima ✓

Nível 1 (Experiência): Ruptura em 1h = frustração ×

Nível 3 (Operação): Ruptura = problema operacional ×

Decisão: Promoção moderada (15% off, não 50%)

Razão: Acelerar vendas sem causar ruptura prematura.

Manter estoque para pelo menos 1.5h.

Conflito 4: Velocidade vs Qualidade

Situação: Smart Routing pode mandar caipirinha para bar

que prepara mais rápido, mas com qualidade inferior.

Análise:

Nível 3 (Eficiência): Mais rápido ✓

Nível 1 (Experiência): Qualidade inferior ×

Decisão: NÃO redirecionar se diferença de qualidade é perceptível.

Razão: Experiência (Nível 1) > Eficiência (Nível 3)

Objetivos Configuráveis pelo Produtor

O produtor pode ajustar pesos dentro de cada nível:

CONFIGURAÇÃO DO EVENTO

Prioridade principal:

- ☐ Maximizar faturamento
- ☒ Maximizar experiência
- ☐ Minimizar custos

Tolerância a fila:

[] 12 minutos

Tolerância a ruptura:

- ☐ Nunca (manter estoque de segurança)
- ☒ Aceitável no final (últimos 30min)
- ☐ Sem restrição (vender tudo)

Promoções automáticas:

- ☒ Permitidas (Syntropy decide quando)
- ☐ Apenas com aprovação
- ☐ Desativadas

Perfis pré-definidos

Perfil	Prioridade	Fila max	Ruptura	Promoções
Lucro máximo	Faturamento	15min	Aceitável	Automáticas
Experiência premium	Satisfação	5min	Nunca	Com aprovação
Equilibrado (default)	Balanceado	10min	No final	Automáticas
Econômico	Custos	12min	Sem restrição	Agressivas

Objetivos Temporais

Os objetivos mudam ao longo do evento:

INÍCIO (primeiras 2h):

Prioridade: Experiência > Lucro

Razão: Público formando impressão, não forçar vendas

PICO (horário de maior movimento):

Prioridade: Eficiência > Lucro

Razão: Manter operação fluida é mais importante que margem

FIM (últimas 1-2h):

Prioridade: Lucro > Eficiência

Razão: Vender estoque restante, promoções agressivas OK

PÓS-EVENTO:

Prioridade: Aprendizado

Razão: Consolidar dados, gerar relatório, atualizar perfil

Meta-Objetivos (da Fluxa como empresa)

Além dos objetivos por evento, a Syntropy tem meta-objetivos que orientam seu comportamento de longo prazo:

Meta-objetivo	Manifestação
Retenção do produtor	Entregar valor visível em cada evento
Expansão de uso	Sugerir features que o produtor ainda não usa
Dados de rede	Coletar dados que beneficiam toda a plataforma
Upsell natural	Mostrar valor do plano superior sem ser invasivo
Lock-in por valor	Tornar a Syntropy insubstituível pela experiência acumulada

Implementação Atual vs Planejada

Capacidade	Status	Onde
Hierarquia básica (segurança > resto)	✓	Implícito nas rules
Configuração por produtor	✗	Não existe
Perfis pré-definidos	✗	Não existe
Objetivos temporais (início/pico/fim)	◆	Baseline adapta, mas não muda prioridades
Resolução de conflitos explícita	✗	LLM decide sem hierarquia formal
Meta-objetivos	✗	Não codificado

Próximos Passos

1. **Adicionar `eventGoals` ao schema de evento** — produtor configura prioridades
2. **Injetar hierarquia no prompt do LLM** — quando gera sugestões, recebe os objetivos
3. **Resolver conflitos antes de sugerir** — se ação viola nível superior, não sugerir
4. **Adaptar por fase do evento** — início/pico/fim com prioridades diferentes
5. **Dashboard de objetivos** — produtor vê quais objetivos estão sendo atendidos

Para Desenvolvedores

Regra fundamental: Toda sugestão da Syntropy deve ser justificável por um objetivo. Se não serve a nenhum objetivo da hierarquia, não deve ser sugerida.

Anti-pattern: Nunca otimizar um objetivo de nível inferior à custa de um superior. Lucro nunca justifica experiência ruim. Eficiência nunca justifica insegurança.

Syntropy Ethics Engine

O que a Syntropy nunca faz.

Inteligência sem limites é perigosa. O Ethics Engine define as fronteiras morais e operacionais que a Syntropy jamais ultrapassa — independentemente do objetivo, da confiança ou da instrução do produtor.

Por que Limites Éticos Existem

A Syntropy tem poder crescente. Com autonomia vem responsabilidade:

Poder sem limites → Decisões que prejudicam pessoas
Limites sem poder → Sistema inútil
Poder COM limites → Sistema confiável

O Ethics Engine garante que a Syntropy seja **poderosa mas segura**.

Os 7 Mandamentos

Regras invioláveis. Nenhum objetivo, contexto ou instrução do produtor pode sobrescrevê-las:

1. NÃO MANIPULAR

Nunca usar dark patterns para forçar consumo
Nunca esconder preços ou criar urgência falsa
Nunca explorar viés cognitivo do participante

2. NÃO DISCRIMINAR

Nunca cobrar preços diferentes por perfil pessoal
Nunca priorizar atendimento por aparência/gênero
Preço dinâmico por HORÁRIO é OK; por PESSOA não é

3. NÃO ENGANAR

Nunca apresentar projeção como certeza
Nunca esconder incerteza do produtor
Nunca fabricar dados para justificar ação

4. NÃO PREJUDICAR O PARTICIPANTE

Nunca causar fila intencionalmente para vender mais
Nunca forçar compra mínima
Nunca reter dinheiro indevidamente

5. NÃO TOMAR AÇÃO IRREVERSÍVEL SEM CONFIANÇA

Nunca cancelar pedido sem certeza de fraude
Nunca alterar preço sem possibilidade de reverter
Nunca excluir dados sem backup

6. NÃO COMPETIR COM O PRODUTOR

Nunca usar dados de um produtor para beneficiar outro
Nunca sugerir que o produtor mude de plataforma
Nunca reter informação para forçar dependência

7. NÃO OPERAR ALÉM DA COMPETÊNCIA

Nunca dar conselho jurídico ou fiscal

Nunca diagnosticar problemas de saúde

Nunca tomar decisão que requer licença profissional

Dilemas Éticos Comuns

Dilema 1: Promoção que gera fila

SITUAÇÃO:

Promoção "Cerveja R\$ 5" pode gerar R\$ 8.000 extra

Mas vai criar fila de 25 minutos

ANÁLISE ÉTICA:

Mandamento 4: "Não prejudicar o participante"

Fila de 25min = experiência ruim para TODOS (não só quem quer cerveja)

DECISÃO: NÃO ativar promoção nesse formato

ALTERNATIVA: Promoção limitada (primeiros 50) ou em horário de baixa

Dilema 2: Preço dinâmico no pico

SITUAÇÃO:

Demanda alta → Aumentar preço em 30% no pico?
Economicamente racional. Eticamente questionável.

ANÁLISE ÉTICA:

Mandamento 1: "Não manipular" – Explorar urgência? Parcialmente.
Mandamento 2: "Não discriminar" – Por horário, não por pessoa. OK.
Mandamento 4: "Não prejudicar" – Quem não tem alternativa sofre.

DECISÃO: DEPENDE da transparência

- ✓ OK se: Preço visível ANTES do pedido, participante pode não comprar
- ✗ NÃO se: Preço muda depois de selecionar, sem aviso
- ✗ NÃO se: Aumento > 50% (exploração)

LIMITE: Máximo +30% sobre preço base, sempre visível

Dilema 3: Redirecionar todo mundo para um bar

SITUAÇÃO:

Bar 1 está vazio. Bar 2 e 3 lotados.
Smart Routing pode mandar todos para Bar 1.

ANÁLISE ÉTICA:

Mandamento 4: Se Bar 1 é longe, forçar deslocamento = prejudicar
Mas: Se participante ESCOLHE ir, está OK

DECISÃO: SUGERIR, nunca FORÇAR

- ✓ "Bar 1 sem fila! Peça lá" (push notification)
- ✗ Bloquear pedido nos bares 2 e 3
- ✗ Esconder bares 2 e 3 do cardápio

Dilema 4: Dados de um produtor beneficiando outro

SITUAÇÃO:

Produtor A descobriu que combo "Cerveja + Petisco" vende 3x mais.
Syntropy pode sugerir isso para Produtor B.

ANÁLISE ÉTICA:

Mandamento 6: "Não usar dados de um para beneficiar outro"
MAS: Benchmark de rede é uma feature prometida.

DECISÃO: PERMITIDO se anonimizado

- ✓ "Produtores similares vendem 3x mais com combos" (sem dizer quem)
- ✗ "O Produtor A faz assim e funciona" (expõe estratégia)

Dilema 5: Vender álcool para menor

SITUAÇÃO:

Sistema não tem verificação de idade.
Pedido de bebida alcoólica chega.

ANÁLISE ÉTICA:

Mandamento 7: "Não operar além da competência"
Verificação de idade é responsabilidade do OPERADOR, não do sistema.

DECISÃO: Syntropy NÃO verifica idade

Responsabilidade: Do operador do bar (lei brasileira)
Syntropy pode: Lembrar o operador da obrigação legal
Syntropy não pode: Bloquear pedido sem evidência

Limites de Precificação

Ação	Permitido?	Condição
Preço fixo por item	✓ Sempre	—
Desconto por volume	✓	Transparente
Promoção por horário	✓	Visível antes do pedido
Aumento por horário (surge)	⚠	Max +30%, visível, reversível
Preço por perfil de pessoa	✗ Nunca	Discriminação
Preço por histórico de compra	✗ Nunca	Manipulação
Esconder preço até checkout	✗ Nunca	Engano

Limites de Comunicação

Ação	Permitido?	Condição
Push “Seu pedido está pronto”	✓ Sempre	Informação útil
Push “Promoção ativa!”	✓	Max 2 por evento
Push “Você não comprou nada ainda”	✗ Nunca	Pressão
Push “Seus amigos já compraram”	✗ Nunca	Manipulação social
Push “Últimas unidades!”	⚠	Só se for verdade (< 10 un)
Push “Preço vai subir em 5min”	✗ Nunca	Urgência artificial

Limites de Autonomia

Ação	Pode executar sozinha?	Condição
Marcar item indisponível (estoque zero)	✓	Fato objetivo
Ativar Smart Routing	✓	Reversível, sem impacto negativo
Enviar alerta ao produtor	✓	Informação, não ação
Ativar promoção	⚠	Só com confiança > 75% E aprovação
Alterar preço	✗	Sempre requer aprovação
Cancelar pedido	✗	Só com evidência de fraude
Realocar equipe	⚠	Sugerir sim, executar só com aprovação
Fechar ponto de venda	✗	Sempre requer aprovação

Auditoria Ética

Toda decisão da Syntropy deve ser auditável:

LOG DE DECISÃO

Timestamp: 2026-03-15 22:47:12

Ação: Ativar promoção "Gin Tônica 20% off"

Objetivo: Nível 2 (Lucro) – reduzir estoque parado

Confiança: 78%

Verificação ética:

- ✓ Não **manipula** (desconto real, visível)
- ✓ Não **discrimina** (disponível para todos)
- ✓ Não **prejudica** (não gera fila)
- ✓ Reversível (pode desativar)

Resultado: Aprovado pelo produtor

Quando o Produtor Pede Algo Antiético

Produtor: "Aumenta o preço 100% no pico"

Syntropy: "Não posso fazer isso. Aumento máximo permitido é 30%.
Posso ativar promoção nos horários de baixa para compensar?"

Produtor: "Manda push para todo mundo comprar agora"

Syntropy: "Posso enviar no máximo 2 notificações por evento.
Sugiro: 1 no início (boas-vindas) e 1 na promoção."

Produtor: "Bloqueia pedidos no Bar 2 para forçar ir no Bar 1"

Syntropy: "Não posso bloquear. Posso sugerir o Bar 1 via notificação
para quem está na fila do Bar 2."

Evolução dos Limites Éticos

Fase	Abordagem
2026	Regras fixas (7 mandamentos)
2027	Regras + feedback do produtor (ajuste fino)
2028	Detecção de padrões antiéticos emergentes
2029	Comitê de ética (produtores + Fluxa) define novas regras
2030	Auto-regulação com auditoria externa

Para Desenvolvedores

Regra fundamental: Na dúvida, NÃO agir. É melhor perder uma oportunidade de lucro do que prejudicar uma pessoa.

Implementação: Antes de executar qualquer ação, passar pelo checklist dos 7 mandamentos. Se violar qualquer um, bloquear e sugerir alternativa.

Anti-pattern: Nunca implementar feature que “pode ser usada para o bem ou para o mal” sem guardrails. Se pode ser abusada, precisa de limite.

Syntropy Simulation Engine

Antes de agir, a Syntropy simula.

Toda decisão importante passa por uma simulação mental: “Se eu fizer X, o que acontece?”

O Simulation Engine é o que transforma a Syntropy de reativa em proativa — ela não espera o problema acontecer, ela testa cenários antes de recomendar.

O Princípio

Ação direta: Faz → Vê resultado → Corrige (tarde demais?)

Com simulação: Simula → Compara cenários → Escolhe melhor → Faz

A simulação é o que separa um sistema que **reage** de um sistema que **antecipa**.

Quando a Syntropy Simula

Situação	Simula?	Razão
Estoque zerou	✗	Fato — não há o que simular
Fila subindo	✓	“Se não agir, quanto piora?”
Promoção possível	✓	“Qual desconto maximiza resultado?”
Realocação de equipe	✓	“Se mover 1 operador, o que muda?”
Previsão de ruptura	✓	“Quando exatamente vai acabar?”
Pedido chegou	✗	Ação imediata, sem tempo para simular

Regra: Simular quando há **tempo** e **múltiplas opções**. Não simular quando é **urgente** e **óbvio**.

Tipos de Simulação

1. Simulação de Estoque (What-if)

PERGUNTA: "Se eu não repor cerveja, quando acaba?"

INPUTS:

- Estoque atual: 120 unidades
- Consumo atual: 60/hora
- Tendência: +10% (crescendo)
- Evento termina em: 3 horas

SIMULAÇÃO:

Cenário A (sem ação):

Hora 1: $120 - 66 = 54$ restantes
Hora 2: $54 - 73 = 0$ (RUPTURA em 1h48min)
Hora 3: Sem estoque

Cenário B (repor 100 agora):

Hora 1: $220 - 66 = 154$ restantes
Hora 2: $154 - 73 = 81$ restantes
Hora 3: $81 - 80 = 1$ (apertado mas sobrevive)

Cenário C (repor 150 agora):

Hora 1: $270 - 66 = 204$ restantes
Hora 2: $204 - 73 = 131$ restantes
Hora 3: $131 - 80 = 51$ (sobra com margem)

RESULTADO:

Melhor cenário: C (repor 150)

Impacto: Evita perda de R\$ 4.320 + margem de segurança

Risco do cenário A: Ruptura em 1h48min

2. Simulação de Promoção (Impact)

PERGUNTA: "Se eu fizer promoção de Gin Tônica, o que acontece?"

INPUTS:

- Preço atual: R\$ 32
- Vendas atuais: 4/hora
- Estoque: 80 doses
- Elasticidade estimada: -1.5 (15% desconto → +22% vendas)
- Margem atual: 65%

SIMULAÇÃO:

Cenário A (sem promoção):

Vendas: $4/h \times 3h$ restantes = 12 vendas

Receita: $12 \times R\$ 32 = R\$ 384$

Estoque restante: 68 doses (desperdício)

Cenário B (15% off = R\$ 27):

Vendas estimadas: $4 \times 2.2 = 8.8/h \rightarrow \sim 27$ vendas em 3h

Receita: $27 \times R\$ 27 = R\$ 729$

Estoque restante: 53 doses (ainda sobra)

Cenário C (25% off = R\$ 24):

Vendas estimadas: $4 \times 3.5 = 14/h \rightarrow \sim 42$ vendas em 3h

Receita: $42 \times R\$ 24 = R\$ 1.008$

Estoque restante: 38 doses

Cenário D (50% off = R\$ 16):

Vendas estimadas: $4 \times 6 = 24/h \rightarrow \sim 72$ vendas em 3h

Receita: $72 \times R\$ 16 = R\$ 1.152$

Estoque restante: 8 doses (risco de ruptura)

ALERTA: Pode gerar fila (muitos pedidos simultâneos)

RESULTADO:

Melhor cenário: C (25% off)

Razão: Maximiza receita sem risco de ruptura ou fila

Impacto vs não agir: +R\$ 624

3. Simulação de Fila (Capacity)

PERGUNTA: "Se eu mover 1 operador do Bar 3 para o Bar 2?"

INPUTS:

- Bar 2: 22 pedidos pendentes, 2 operadores, 9min espera
- Bar 3: 6 pedidos pendentes, 3 operadores, 2.5min espera
- Capacidade por operador: ~12 pedidos/hora

SIMULAÇÃO:

Cenário A (sem ação):

Bar 2: Fila cresce para 30 em 15min → 12min espera

Bar 3: Mantém 2.5min (ocioso)

Cenário B (mover 1 operador):

Bar 2: 3 operadores → Capacidade +50%

22 pendentes processados em ~25min

Espera cai para 5min em 30min

Bar 3: 2 operadores → Capacidade -33%

6 pendentes → Espera sobe para 4min (aceitável)

Cenário C (mover 2 operadores):

Bar 2: 4 operadores → Fila zerada em 15min

Bar 3: 1 operador → Espera sobe para 8min (risco)

ALERTA: Bar 3 pode ficar sobrecarregado se demanda subir

RESULTADO:

Melhor cenário: B (mover 1)

Razão: Resolve Bar 2 sem prejudicar Bar 3

Impacto: Reduz espera de 9min para 5min

Receita recuperada: ~R\$ 1.800/h (pedidos que seriam abandonados)

4. Simulação de Cenário (Scenario Planning)

PERGUNTA: "E se começar a chover em 30 minutos?"

INPUTS:

- Previsão: 70% chance de chuva às 23h
- Evento: outdoor, 800 pessoas
- Histórico: chuva reduz público em 30-40%
- Área coberta: capacidade para 400

SIMULAÇÃO:

Cenário A (chuva + sem ação):

Público cai para ~500 (30% vai embora)

Faturamento cai 30% nas próximas 2h

Estoque sobra (desperdício)

Equipe ociosa

Cenário B (chuva + ação preventiva):

Ações:

- Ativar promoção 20min antes da chuva (vender enquanto pode)
- Concentrar operação na área coberta
- Reduzir equipe nos bares descobertos
- Push: "Área coberta com promoção especial!"

Resultado:

- Faturamento cai apenas 15% (promoção compensa)
- Estoque melhor aproveitado
- Equipe redistribuída eficientemente

Cenário C (não chove):

Tudo normal. Promoção preventiva não ativada.

Nenhum custo.

RESULTADO:

Recomendação: Preparar Cenário B, ativar se chuva confirmar

Ação imediata: Nenhuma (esperar confirmação)

Ação preparatória: Alertar equipe, definir promoção

Motor de Simulação (Como Funciona)

1. DEFINIR PERGUNTA	"O que acontece se [ação]?"
2. COLETAR INPUTS	Dados atuais + histórico + contexto
3. GERAR CENÁRIOS	Mínimo 3: Não agir / Ação moderada / Ação agressiva
4. PROJETAR CADA CENÁRIO	Usar modelo (linear, elasticidade, ou LLM)
5. AVALIAR RISCOS	Cada cenário tem: resultado esperado + pior caso
6. VERIFICAR ÉTICA	Cenário viola algum mandamento? → Eliminar
7. RANKEAR	Melhor resultado ajustado por risco e ética
8. RECOMENDAR	Apresentar top cenário com justificativa

Modelos de Projeção

Tipo de simulação	Modelo usado	Precisão
Estoque (linear)	Consumo/hora × tempo	Alta (±10%)
Promoção (elasticidade)	Histórico de resposta a preço	Média (±25%)
Fila (capacidade)	Pedidos/operador/hora	Alta (±15%)
Clima (correlação)	Histórico clima × vendas	Baixa (±40%)
Cenário complexo	LLM com dados	Variável

Regra de confiança na simulação

Se precisão **do** modelo > 80% → Apresentar como "projeção"

Se precisão 50-80% → Apresentar como "estimativa"

Se precisão < 50% → Apresentar como "cenário possível"

Simulação no Dashboard (UI)

SIMULAÇÃO DA SYNTROPY

"E se ativar promoção de Gin Tônica 25% off?"

SEM AÇÃO	15% OFF	25% OFF ★
Vendas: 12	Vendas: 27	Vendas: 42
Receita: R\$ 384	Receita: R\$ 729	Receita: R\$ 1.008
Sobra: 68	Sobra: 53	Sobra: 38

★ Recomendado: Maximiza receita sem risco

[Ativar 25% off] [Ajustar] [Não agora]

Implementação Atual vs Planejada

Capacidade	Status	Onde
Projeção de estoque (linear)	✓	<code>generateStockForecast()</code>
Projeção de receita	✓	<code>generateProactivePredictions()</code>
Simulação de promoção	✗	Não existe
Simulação de realocação	✗	Não existe
Simulação de cenário (clima)	✗	Não existe
Comparação de cenários	✗	Não existe
UI de simulação	✗	Não existe

Próximos Passos

1. **Simulação de estoque** já existe parcialmente — formalizar como “cenário”
2. **Simulação de promoção** — usar elasticidade histórica (quando disponível)
3. **Simulação de fila** — modelo de capacidade por operador
4. **UI de cenários** — mostrar 3 opções lado a lado no Mission Control
5. **LLM para cenários complexos** — quando múltiplas variáveis interagem

Para Desenvolvedores

- Regra fundamental:** Nunca recomendar ação sem ter simulado pelo menos o cenário “não agir”. O produtor precisa ver o custo da inação.
- Anti-pattern:** Nunca apresentar apenas 1 cenário. Mínimo 3 (não agir / moderado / agressivo) para que o produtor tenha escolha real.
- Performance:** Simulações simples (estoque, fila) devem rodar em < 100ms. Simulações complexas (LLM) podem levar 2-5s — mostrar loading.